

Homework 2

请完成以下练习题目，算法设计只需要用**伪代码方式**书写，不需要实际编程。

1. 设 $A.elem[m]=(a_1, \dots, a_m)$ 和 $B.elem[n]=(b_1, \dots, b_n)$ 均为顺序表， A' 和 B' 分别为 A 和 B 中除去最大共同前缀后的子表（例如， $A=(x,y,y,z,x,z)$ ， $B=(x,y,y,z,y,x,x,z)$ ，则两表中最大的共同前缀为 (x, y, y, z) ，在两表中除去最大共同前缀后的子表分别为 $A'=(x, z)$ 和 $B'=(y, x, x, z)$ ）。若 $A'=B'$ =空表，则 $A=B$ ；若 A' =空表，而 $B' \neq$ 空表，或者两者均不为空表，且 A' 的首元小于 B' 的首元，则 $A < B$ ；否则 $A > B$ 。试写一个比较 A, B 大小的算法。（注：在算法中，不要破坏原表 A 和 B ，并且，也不一定先求得 A' 和 B' 才进行比较）。

思路：用同一个下标变量 i 控制两个线性表，逐个比较对应元素是否相同，直到对应元素不同，或某个表已经到结尾。

```
int CompareSqlint(Sqlink A, Sqlink B)
{ //比较顺序表 A、B 的大小，若 A>B，则返回 1；
  //A<B，则返回-1；若 A==B，则返回 0
  int i=0;
  if(A.length==0&&B.length==0) return 0;
  //顺序表 A、B 都为空表，则相等
  while(i<A.length&&i<B.length&&A.elem[i]==B.elem[i])
    i++;
  if(A.elem[i]>B.elem[i]) return 1;
  if(A.elem[i]<B.elem[i]) return -1;
  if(i==A.length&&i==B.length) return 0;
  else if(i==A.length) return -1;
  else return 1;
}
```

2、请用类 C/C++ 伪代码，写出一个交换单链表中两个结点位置的算法（自定义变量类型）。

本题意为交换单链表中指针 **p** 所指结点和其后继结点，令指针 **head** 为该链表的头指针，指针 **p** 为指向该链表中要交换结点位置的一个结点。若指针 **p** 所指的结点存在后继结点，看它是否是头结点，如果是，则交换后还要改变该链表的 **head**，若不是头结点，则直接交换。

参考实现如下：

```
A1. q = p->next;
A2. if q != null
    {
        if p = head
        {   s = head.next; head.next = p; p.next = s;   }
        else
        {   r = head;
            while (r.next != p)
                r = r.next;
            r.next = q; p.next = q.next; q.next = p;
        }
    }
    else printf('no next node');
A3. End
```

3、请用类 C/C++ 伪代码，试编写算法，将一个用循环链表表示的稀疏多项式分解成两个多项式，使这两个多项式中各自仅含奇次项或偶次项，并要求利用原链表中的结点空间构成这两个链表。

基本思想是在遍历表的过程中，将结点分为两类而插入到两个链表中，具体实现算法如下：

struct PNode{ // 结点类型

```
    float coef;
    int exp;
    struct PNode *next;
```

} Pnode, *LinkList;

void Split(Linklist &plyn, LinkList &odd)

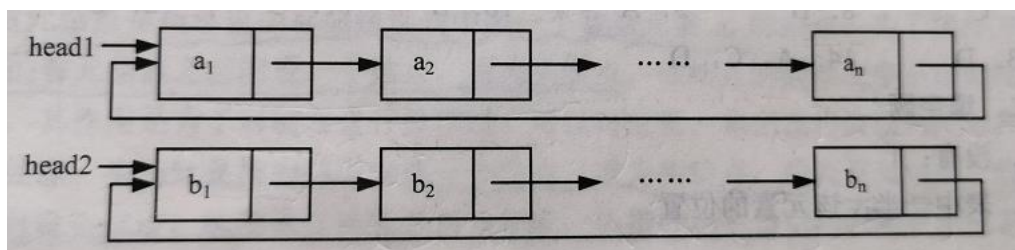
```
{
    // plyn 指向稀疏多项式的循环链表头结点，odd 为新产生的
    // 仅含奇次项链表的头指针，运算后 plyn 链表中仅含偶次项
    LinkList p, q, s;
    odd = (LinkList) malloc(sizeof(PNode));
    odd->next = odd; // 建立奇次项循环空链表
```

```

q = plyn; p = plyn->next; s = odd;
while (p!=plyn)
{
    if (p->exp%2==0)
    { q = p; p = p->next; }
    else
    {
        q->next = p->next; // 从原表中删去奇次项结点
        p->next = s->next;
        s->next = p; s = s->next; // 插入至新表中
        p = q->next;
    }
}
}
}

```

4、有两个循环单链表，链表头指针分别为 head1 和 head2，如下图所示。编写一个函数将链表 head2 链接到链表 head1 之后，链接后的链表仍然保持是循环链表的形式。



19. 算法基本思想是：先找到两链表的尾指针，将一个链表的尾指针与另一个链表的头结点链接起来，再使之成为循环。设循环链表的结构如下：

```

typedef struct DuLNode{
    Elemtype data;
    struct DuLNode *prior;
    struct DuLNode *next;
}DuLNode, *DuLinkList;

```

实现算法如下：

```

DuLinkList link(DuLinkList head1,DuLinkList head2)
{
    DuLinklist p,q;
    p=head1;
    while(p->next!=head1)          //找 head1 的表尾 p
        p=p->next;
    q=head2;
    while(q->next!=head2)          //找 head2 的表尾 q
        q=q->next;
    //将一个链表的尾指针指向另一链表的头结点，使之成为循环链表
    p->next=head2;  q->next=head1;
    return(head1);
}

```